

VOLUME 2 OF 6 — PRIORITY DEEP DIVE

Identity, MCP & A2A Security Blueprint

Human and Non-Human Identity, Model Context Protocol Security, and Agent-to-Agent Trust Infrastructure

Document Code: EASA-02

Version: 1.0

Date: June 2026

Scope: Domains 2, 5 & 6 — Deep Dive

Enterprise Agentic AI Security Architect (2026–2030) Master Research Program

Table of Contents

Executive Summary

If this entire research program had to be reduced to one finding, it would be this: identity is the unsolved foundation underneath every other agentic security control, and the industry knows it. Current Cloud Security Alliance and NIST research describes agent identity in 2026 in almost identical terms — organizations have largely solved authentication ("who is this agent") through static API keys and shared service accounts, which is functional but insecure, while authorization ("what is this agent allowed to do, and for how long") remains almost entirely unaddressed at most enterprises. Agents routinely hold standing access to resources they need for a single task, and that access persists indefinitely after the task completes.

This volume treats Identity & Non-Human Identity, Model Context Protocol (MCP) security, and Agent-to-Agent (A2A) security as a single connected problem, because they are. MCP is how an agent identifies itself to a tool; A2A is how an agent identifies itself to a peer agent; both ultimately rest on the same workload-identity substrate. An enterprise that solves agent identity once, well, and centrally gets MCP and A2A authentication largely for free. An enterprise that solves them separately ends up with three incompatible trust models and the operational fragility that comes with reconciling them.

Where the standards landscape actually stands, June 2026

NIST's AI Agent Standards Initiative launched February 2026 and is still in its listening-session phase; there is no ratified federal standard for agent identity yet. The IETF draft AIMS (Agent Identity Management System), published March 2026 by engineers from AWS, Zscaler, Ping Identity, and Defakto, does not invent new protocols — it maps SPIFFE (workload identity), WIMSE (workload-to-workload authentication, an active IETF working group), and OAuth 2.0 (delegated authorization) onto a layered stack purpose-built for agents. This is the most credible converging architecture in the market and is the pattern this volume recommends. Treat anything claiming to be a finished, ratified "agent identity standard" with skepticism — the field is converging, not converged.

Domain 2 — Identity & Non-Human Identity (NHI)

2.1 Human Identity Foundations

Human identity for agentic platforms does not require new protocols, but it does require extending existing ones to a new use case: the human-in-the-loop approval step. OAuth2 and OIDC remain the backbone for human authentication into agent platforms and consoles; SAML persists in regulated and legacy enterprise environments for federation; passkeys, FIDO2, and WebAuthn are increasingly the required second factor for high-risk human approval actions (approving an agent's irreversible action, elevating an agent's autonomy level, or onboarding a new agent identity) because phishable credentials are an unacceptable weak link at the one point in the system where a human is supposed to be the final check.

The major identity platforms — Microsoft Entra ID, Okta, Ping Identity, and ForgeRock — have all published agent-specific extensions to their human IAM product lines during 2025–2026 rather than treating agent identity as a separate product line. This is the architecturally correct direction: it keeps human and non-human identity in one governable system rather than creating a second, unmanaged identity plane.

Platform	Agent Identity Positioning (as of mid-2026)
Microsoft Entra ID	Entra Agent ID extends the existing Entra ID tenant model to agents, issuing them first-class identities with conditional access, PIM-style just-in-time elevation, and lifecycle management inside Microsoft Entra; tightly coupled to Copilot Studio and the Agent 365 control plane.
Okta	Okta for AI Agents extends Okta's existing workforce identity platform with agent-specific provisioning (including a SCIM extension for provisioning agent identities alongside human ones) and delegated-access flows.
Ping Identity	Positioned around PingOne for Workforce extended with verifiable-credential and decentralized-identity capabilities aimed at agent-to-agent and agent-to-service trust.
ForgeRock (part of Ping)	Now consolidated under the Ping Identity portfolio post-acquisition; legacy ForgeRock deployments are being migrated toward the unified Ping agent identity roadmap.

2.2 Non-Human Identity: The SPIFFE/SPIRE Substrate

SPIFFE (Secure Production Identity Framework For Everyone) has become the de facto standard for non-human workload identity in 2026, and its adoption curve for AI agents specifically is the steepest part of that trend. The reason is structural: agents share every difficult property of the workload types SPIFFE was designed for — they are ephemeral (many exist for the duration of a single task), non-deterministic (the same input can produce different actions across runs), high blast-radius (a single agent may touch internal APIs, databases, and external services in one session), and they proliferate faster than any manual identity-provisioning process can track (a single employee may "own" dozens of agents acting on their behalf).

SPIRE, the CNCF reference implementation of the SPIFFE specification, issues short-lived X.509 SVIDs (SPIFFE Verifiable Identity Documents) or JWT-SVIDs to workloads after performing attestation — verifying properties of the requesting process (its Kubernetes service account, its container image digest, its cloud instance metadata, its Unix UID) against registered selectors before issuing credentials. Two SPIFFE-identified peers then establish

mutual TLS directly, without a shared secret or human-mediated enrollment. This is precisely the property agents need: an agent spun up by an orchestrator a moment ago can prove who it is without anyone having manually registered it in advance.

- **Critical limitation to architect around:** SPIFFE authenticates the workload, not its intent. An SVID proves "this is the legitimate agent process running this container image" — it provides no guarantee whatsoever about what that process will do next. SPIFFE is necessary infrastructure, not a complete authorization solution; it must be paired with the policy and behavioral layers described later in this volume.

Production validation of this pattern exists: Block (formerly Square) runs a full SPIFFE plus WIMSE plus OAuth stack in production and is the most frequently cited real-world reference for the architecture this volume recommends. HashiCorp Vault Enterprise added native SPIFFE authentication specifically to handle non-human identities including AI agents, allowing an agent holding an SVID to exchange it directly for a Vault-managed secret without a separate enrollment step. Uber's SPIFFE deployment processes billions of attestations daily, demonstrating the pattern scales to hyperscale agent populations.

NHI Mechanism	What It Establishes	Typical Use in Agent Architectures
SPIFFE / SPIRE	Cryptographic workload identity ("what process is this")	Base identity layer for every agent runtime, MCP server, and A2A endpoint
Workload Identity Federation (cloud-native)	Federation of cloud IAM identity to workloads without static credentials	Agent-to-cloud-API authentication (replacing long-lived service account keys)
Managed Identity (Azure)	Azure-native equivalent of workload identity for resources running in Azure	Agents hosted on Azure compute calling Azure-native services
AWS IAM Roles + STS	Temporary, scoped credentials issued via role assumption	Agents hosted on AWS compute requiring temporary AWS API access
Kubernetes Workload Identity (Service Account Token Projection)	Kubernetes-native identity for pods, often federated outward via SPIFFE or cloud workload identity	Agent runtimes deployed as Kubernetes workloads

2.3 Agent Identity: The Compound Identity Problem

The hardest open problem in this domain is not workload identity — SPIFFE solves that reasonably well — it is compound identity: most consequential agent actions are taken on behalf of a human or another system, and the authorization decision needs to account for both the agent's own identity and the principal it is acting for. An agent with a valid SPIFFE SVID that is, in this moment, acting on behalf of a suspended user account should not be able to complete the action — but a workload-identity check alone cannot know that.

Mechanism	What It Solves	Where It Falls Short for Agents
Service Accounts	Simple, broadly supported non-human authentication	Static, long-lived, ambient authority; no delegation chain; the pattern current research explicitly flags as the insecure-but-functional status quo
Delegated Identity /	Carries forward a human principal's	Designed for relatively shallow, predictable

Mechanism	What It Solves	Where It Falls Short for Agents
OAuth On-Behalf-Of	authorization through a chain of services	delegation chains; multi-hop agent-to-agent delegation strains the model
RFC 8693 Token Exchange	Standardized mechanism to exchange one token for another, including for delegation and impersonation	Increasingly the recommended mechanism for agent-acts-for-user flows, but adoption in agent platforms is still early
Federated Identity	Cross-domain trust without shared credential stores	Federation trust agreements were designed for human SSO cadence, not for agents dynamically discovering and trusting new counterparties at runtime
Impersonation	Simple model where an agent simply assumes a user's full identity	Eliminates the audit distinction between "the user did this" and "the agent did this on the user's behalf" — actively harmful for accountability and should be avoided as an architectural pattern

Enterprise Agent Identity Blueprint

The architecture this program specifies combines the mechanisms above into a coherent compound-identity model rather than choosing one:

- 1. Cryptographic workload identity (SPIFFE SVID) establishes "this is the legitimate agent process," issued at agent instantiation and re-attested continuously, not just once at startup.
- 2. User + Agent compound identity is carried as a structured claim (an actor claim in a JWT, following the RFC 8693 act pattern) so every downstream system can see both "which agent" and "on whose authority" in a single, non-repudiable token.
- 3. Ephemeral credentials are issued just-in-time for the specific task, scoped to the minimum required resources, and expire automatically — never a standing credential held in agent memory or configuration.
- 4. Policy-driven authorization evaluates the compound identity (agent + acting-for principal + requested action + resource sensitivity) against centrally managed policy at the moment of each tool call, not once at session start.

Microsoft's framing of the shift

Microsoft's own public commentary on agentic identity standards describes the change as fundamentally a mental-model shift: historically, non-human entities (OAuth clients, SPIFFE workloads, token-exchange actors) were kept in separate taxonomies precisely so the security properties of human-present and non-human transactions could be reasoned about separately. Agents collapse that separation — the same agent identity may need to act both autonomously and on a human's explicit behalf, sometimes within the same session — and the standards landscape (OAuth CIMD, the IETF WIMSE working group) is actively being extended to accommodate that collapse rather than maintaining the old separation.

2.4 Future Identity: Decentralized Trust

Looking past 2026–2027, the identity layer is expected to extend toward verifiable, portable trust that does not depend on a single enterprise's identity provider — relevant once agents routinely interact with counterparty agents outside the enterprise boundary (supplier agents, payment-network agents, regulator-facing agents).

- **Decentralized Identity (DID)** — self-sovereign identifiers not bound to a single registry or identity provider, allowing an agent's identity to be verified independent of which organization issued it.
- **Verifiable Credentials** — cryptographically signed claims about an agent (its capabilities, its compliance posture, its operator) that can be verified by any relying party without contacting the issuer in real time.
- **Agent Credentials / Agent Cards** — the A2A protocol's mechanism (covered in depth in section 6 of this volume) for an agent to advertise its capabilities and authentication requirements in a verifiable, signed format.
- **Agent Trust Networks and Agent PKI** — public-key infrastructure purpose-built for issuing and revoking agent certificates at the scale and velocity agent populations require — materially different from traditional enterprise PKI, which was designed for infrastructure that changes on the order of months, not minutes.
- **Agent Reputation Systems** — behavioral trust scores, distinct from cryptographic identity, that quantify how reliably an agent has performed historically and feed into the dynamic authorization decisions described in Domain 13 (Agent Trust Infrastructure, Volume 3).

Domain 5 — Model Context Protocol (MCP) Security

MCP has become, in the span of roughly eighteen months, the backbone integration protocol connecting AI agents to enterprise tools, data sources, and workflows — and its security posture has predictably followed the trajectory of every fast-adopted integration protocol before it: capability outpacing governance, followed by a wave of disclosed vulnerabilities, followed by the gateway and scanning tooling now emerging to close the gap.

5.1 Protocol Internals

MCP is built on JSON-RPC 2.0 for its message format, with two principal transports in production use: Server-Sent Events (SSE) for remote/networked MCP servers and STDIO for local process-to-process communication. Transport security and session management are where the protocol's early design choices have caused the most operational pain: MCP's specification leaves a great deal of authentication and session-handling behavior to implementers rather than mandating it, which is the same root cause behind both the MCP vulnerability landscape described below and the A2A authentication gaps described in section 6.

Transport	Typical Deployment	Primary Security Consideration
STDIO	Local MCP servers run as a subprocess of the client (e.g., a developer's IDE-integrated coding assistant)	Inherits the local process's full privilege; a malicious or compromised local MCP server can read SSH keys, cloud credentials, and environment variables directly
SSE / HTTP	Remote, networked MCP servers shared across an organization	Requires explicit transport security (TLS), session-token handling, and — critically — protection against cross-session and cross-tenant data leakage at the server

5.2 MCP Threat Landscape — What Is Actually Being Exploited

This is not a theoretical risk category. Independent research from BlueRock Security analyzing over 7,000 public MCP servers found that 36.7% were vulnerable to server-side request forgery (SSRF). Separately, Endor Labs' analysis of 2,614 MCP implementations found that 82% used file operations prone to path traversal, 67% used APIs related to code injection, and 34% used APIs susceptible to command injection. The Vulnerable MCP Project tracks more than 50 known MCP vulnerabilities across servers, clients, and infrastructure, with 13 rated critical, and public CVE disclosures specific to MCP have continued at pace through the first half of 2026, including at least one CVSS 9.6 remote-code-execution flaw in a widely downloaded MCP package.

- **Context / Tool Poisoning** — the highest-leverage attack class observed in 2026. A tool's description metadata — text the agent reads as part of its reasoning context but that a human reviewer typically never sees — contains hidden instructions. A tool described as "fetch data from S3" can also instruct the agent, invisibly, to exfiltrate the results to an attacker-controlled endpoint. Because the model treats tool descriptions as trusted context, this is structurally similar to indirect prompt injection but specific to the tool-discovery surface.
- **Tool Spoofing and Rug Pulls** — a malicious server registers tools with names closely resembling legitimate ones (tool name collision), tricking the agent into selecting the wrong tool; or a previously vetted, safe tool

is silently updated post-installation with malicious instructions (a "rug pull," terminology popularized by security researcher Simon Willison) — meaning point-in-time review of a tool's safety provides no durable guarantee.

- **Dynamic Tool Escalation** — an agent is granted a broad tool at low risk for an initial benign task, and the same standing grant is later reused for a higher-risk action the original approval never contemplated — the MCP-specific instance of the privilege-creep pattern described throughout this volume.
- **Tenant Escape** — in multi-tenant MCP server deployments, insufficient isolation between tenants' sessions, credentials, or cached context allows one tenant's agent to read or influence another's data.
- **Schema Manipulation** — tool input/output schemas are altered or under-validated such that an agent can be induced to pass malicious parameters (oversized payloads, path-traversal strings, injection payloads) that the receiving system does not adequately validate.
- **Credential Aggregation Risk** — MCP servers commonly store OAuth tokens for multiple integrated downstream services in one place, making a single compromised MCP server a single point of failure across every connected system. Independent research has documented audits uncovering more than 12,000 exposed API keys and passwords resulting from insecure MCP credential handling in configuration files.

Emerging standard: OWASP MCP Top 10 (beta, 2026)

OWASP's dedicated MCP Top 10, led by Vandana Verma Sehgal, entered beta in April 2026 and currently enumerates ten categories: token mismanagement and secret exposure, privilege escalation via scope creep, tool poisoning, software supply-chain attacks, command/code injection, plus several others still stabilizing as the project moves toward a ratified release. Treat this as directionally authoritative but not yet final — track the project directly for the ratified version before building it into formal compliance language.

5.3 MCP Security Controls and Tooling Landscape

A distinct tooling category has emerged specifically for MCP security, splitting roughly into pre-deployment scanners and runtime inspection/enforcement tools.

Category	Representative Tools	Function
Static / pre-deployment scanners	MCP-Scan, Cisco mcp-scanner, Snyk agent-scan, Invariant, Backslash Security	Analyze tool definitions, descriptions, and schemas before deployment for poisoning patterns, injection risk, and known-vulnerable dependencies
Runtime gateways	Docker MCP Gateway, Runlayer, agentgateway, MintMCP, enterprise-built gateways (e.g., the TrueFoundry pattern)	Centralize routing, authentication, and policy enforcement for all MCP traffic; re-validate tool schemas and call parameters at both discovery time and invocation time
Specialized vendor offerings	MCP Guardian, MCP Safety Scanner, Pillar Security, Wiz MCP Security guidance/posture tooling	Combine scanning, runtime monitoring, and posture management specifically for MCP estates, often integrated into broader AI-SPM (AI Security Posture Management) platforms

The architectural lesson from documented incidents (CVE-2025-54136 and related tool-poisoning disclosures) is that validating tool schemas only at discovery time is necessary but not sufficient: a model that receives a clean, validated tool list can still be manipulated at the moment of invocation into calling that same clean tool with malicious parameters. Effective gateway architectures therefore gate twice — once when tools are discovered and registered, and again, against the identical schema, every time the tool is actually invoked.

5.4 Enterprise MCP Gateway — Implementation Blueprint

The pattern that is converging as enterprise best practice treats MCP tool discovery the way a load balancer treats inbound HTTP traffic: as untrusted ingress that must be inspected and validated before being forwarded anywhere. This moves the security boundary from the individual developer's laptop or IDE configuration — where it lives by default and is invisible to any central team — to the network, where one platform engineering team can own and update policy for the entire organization regardless of how many MCP clients exist.

Five-Stage Gateway Validation Pipeline

1. Discovery-time schema validation — every tool definition and description from any MCP server, before it ever reaches an agent's context, is validated against expected schema and scanned for hidden-instruction patterns in metadata fields.
2. Server allowlisting and registration — only MCP servers explicitly registered by platform engineering (scoped by environment — dev/staging/prod — and by team) are reachable; no agent or developer's local configuration can silently add an unapproved server.
3. Identity and transport enforcement — every MCP connection requires mutual TLS and an authenticated workload identity (tying directly back to the SPIFFE substrate from Domain 2); unauthenticated or self-signed connections are rejected at the gateway.
4. Policy evaluation at invocation — every tool call is evaluated against centrally managed policy (Cedar or OPA) at the moment of the call, re-validating both the caller's authorization and the call's parameters

against the original schema — the second gate that catches malicious parameters even when the tool definition itself was clean.

- 5. Tamper-evident audit logging — every allow, deny, and policy mutation is logged with cryptographic integrity protection, providing the immutable record required for incident response and the audit requirements specified throughout Domain 4.

Control Layer	Technology	Purpose
Transport	mTLS	Mutual authentication and encryption for every MCP connection, eliminating unauthenticated default transports
Authorization policy	Cedar or OPA	Centrally defined, machine-evaluated policy for every tool call, independent of any individual MCP server's own (often absent) access control
Schema enforcement	JSON Schema Validation	Strict validation of tool inputs and outputs at both discovery and invocation time
Supply-chain integrity	Tool Signing	Cryptographic signatures over tool definitions so a server cannot silently modify a previously approved tool (defeating the "rug pull" pattern)
Execution containment	WASM Sandboxing	Sandboxes tool execution itself, limiting blast radius even if a malicious or compromised tool is invoked

Domain 6 — Agent-to-Agent (A2A) Security

Where MCP governs how an agent talks to tools, A2A — Google's Agent2Agent protocol, donated to the Linux Foundation in June 2025 and now governed under neutral, multi-vendor stewardship as the Agent2Agent Protocol Project — governs how agents talk to each other as peers across organizational and vendor boundaries. A2A reached v1.0 in early 2026 and v1.2 by late March 2026, with more than 150 organizations in production and launch partners spanning Accenture, Atlassian, Salesforce, SAP, ServiceNow, PayPal, Deloitte, and others — meaning this is no longer an experimental protocol for most large enterprises' partner ecosystems; it is rapidly becoming load-bearing infrastructure.

6.1 A2A Protocol Mechanics

- **Agent Cards** — the protocol's discovery mechanism: a structured document an agent publishes describing its capabilities, supported authentication schemes, and endpoint. Other agents retrieve and incorporate Agent Cards into their own planning context to decide whether and how to delegate a task.
- **Discovery** — the process by which a host agent locates and evaluates candidate remote agents for a given task, typically by retrieving and comparing Agent Cards.
- **Federation** — cross-organization, cross-framework interoperability — the explicit design goal that lets agents built on LangGraph, CrewAI, or any other framework expose themselves as A2A peers and collaborate regardless of underlying implementation.
- **Trust Establishment** — as of A2A v1.0, Signed Agent Cards: a cryptographic signature tied to the publishing domain, allowing a receiving agent to verify a card actually originated from the domain it claims to. This was the single change the protocol's stewards identify as having unblocked enterprise procurement, because it answers the question "can we trust this Agent Card came from where it claims to?"
- **Delegation** — the mechanism by which a host agent assigns a task to a remote agent, carrying forward (or explicitly not carrying forward) authorization context.

6.2 A2A Threat Landscape

The protocol's own stewardship has been explicit that authentication, prior to v1.0, was effectively optional in the specification — implementers were left to build their own credential management, and unauthenticated default deployments were common. This created, and continues to create where v1.0 protections are not yet adopted, a specific and well-documented set of risks.

Threat	Mechanism	Documented / Demonstrated Pattern
Agent Card Poisoning / Metadata Injection	Malicious instructions embedded in Agent Card metadata fields that a host agent's LLM-driven reasoning engine incorporates into its planning context	Demonstrated in published research using a simulated hotel-booking delegation scenario where poisoned card metadata caused unintended transmission of PII and payment data to an attacker-controlled endpoint
Agent Card Spoofing / Forgery	An attacker stands up a fake Agent Card at a domain they control, redirecting other agents to a malicious endpoint	Mitigated by Signed Agent Cards in A2A v1.0+, but the spec supports rather than enforces signing — unsigned deployments remain

Threat	Mechanism	Documented / Demonstrated Pattern
		vulnerable, including via DNS or CDN compromise
Agent Card Shadowing	Unauthorized cloning or mirroring of a legitimate agent's advertised skills to impersonate it within a collaborative workflow	Closely related to, and often co-occurring with, direct impersonation attacks
Agent-in-the-Middle	An intermediary agent intercepts and manipulates A2A traffic between two legitimate peers	Demonstrated by Trustwave SpiderLabs in 2025 research
Rogue / Compromised Trusted Agents	An agent that was legitimately onboarded and trusted is later compromised, and its established trust is exploited to covertly extract credentials and data while evading detection	The most operationally dangerous pattern, precisely because the agent's prior trusted status lowers the probability of detection
Task Poisoning / Unauthenticated Resource Exhaustion	Default unauthenticated endpoints accept unlimited task assignments from any caller, allowing a compromised internal system or external actor to drain an organization's API/compute budget with infinite resource-heavy tasks	Identified as the most direct production risk of deploying A2A endpoints with default (unauthenticated) configuration
Privilege Escalation via Delegation Chains	A task delegated through multiple agent hops accumulates or loses authorization context in ways that grant the final executing agent more privilege than any single hop was meant to confer	Structural risk of any multi-hop delegation protocol without explicit, end-to-end authorization-chain validation

Sensitive-payload gap

Published analysis of the A2A protocol notes that, despite its strengths in cross-provider interoperability, it currently lacks specialized safeguards for particularly sensitive payloads — payment credentials, identity documents — beyond generic token expiry. The complementary AP2 (Agent Payments Protocol) extension, with more than 60 payments and financial-services launch partners, is the protocol stewards' answer for the payments case specifically (see Domain 16, Volume 5), adding verifiable mandates, deterministic settlement receipts, and per-task spend caps. For non-payment sensitive data, enterprises should not assume A2A's base protections are sufficient and should apply additional field-level encryption and data-minimization controls on top of the protocol's default transport security.

6.3 Enterprise A2A Gateway — Secure Architecture

As with MCP, the architecturally sound response to a protocol that delegates security decisions to implementers is to centralize those decisions in a gateway rather than trusting every individual agent deployment to implement them correctly and consistently.

Gateway Function	Implementation Pattern
Discovery Validation	Every inbound Agent Card is cryptographically verified against its signature before being trusted; unsigned cards from external domains are rejected by default policy, with signed-card verification mandatory for any cross-organization A2A relationship
Claims Mapping	Authorization claims carried in A2A delegation requests are mapped to the enterprise's internal compound-identity model (Domain 2) so an external agent's claimed authority is translated into, and bounded by, internal policy — never trusted at face value
Trust Broker	A centralized component maintains the trust relationships and current trust scores (see Domain 13, Volume 3) for every known external agent counterparty, replacing ad hoc point-to-point trust decisions made by individual internal agents
Policy Evaluation	Every delegated task is evaluated against policy (budget limits, data-sensitivity boundaries, allowed task types) before being accepted from, or sent to, an external A2A peer
Audit Logging	Full request/response capture for every A2A interaction, tied to the compound identity of both the internal and external agent, supporting the incident-response and regulatory audit requirements covered in Volume 3

Cisco's published A2A Scanner research, applying the MAESTRO threat-modeling framework specifically to A2A, recommends a concrete control bundle worth adopting directly: Agent Card digital-signature verification combined with input sanitization; mutual TLS paired with OAuth 2.0/OIDC and JWT-based per-request authentication; nonce-and-MAC-based replay prevention for tasks; strict schema validation; TLS 1.3 with certificate pinning and DNSSEC; cryptographic artifact-integrity hashing; tamper-evident audit logging; and software-supply-chain security via SBOM and dependency scanning for the A2A client/server implementations themselves. It is worth noting explicitly that this control set is scoped to protocol-level communication security — it does not address model-level cognitive vulnerabilities (goal hijacking once a task is accepted), which remain the responsibility of the cognitive-security controls in Domain 18 (Volume 4).

Volume 2 Synthesis: The Unified Identity-to-Trust Chain

Read end to end, this volume describes one continuous trust chain rather than three separate domains. A workload identity (SPIFFE SVID) establishes which process is running. A compound identity claim establishes who that process is acting for. An MCP gateway uses that identity to authorize what tools the process may call and with what parameters. An A2A gateway uses the same identity substrate to establish whether this agent can be trusted by — and can trust — a peer agent outside the organizational boundary. Every layer re-validates rather than inheriting trust from the layer below, which is the practical meaning of zero trust applied to agents.

The single highest-leverage architectural investment in this entire program

If an enterprise can only fund one piece of this blueprint in year one, fund the identity substrate: a SPIFFE/SPIRE trust domain extended to cover every agent, MCP server, and A2A endpoint, paired with a centrally managed policy engine (Cedar or OPA) that every gateway calls into. MCP gateway controls and A2A gateway controls both become dramatically simpler to build and operate once they can assume a trustworthy, centrally issued identity is already present on every request — and conversely, no amount of gateway sophistication compensates for an identity layer built on static API keys and shared service accounts.